LAPORAN PRAKTIKUM – 5

ANALISIS KOMPLEKSITAS ALGORITMA

DISUSUN OLEH :

NAMA	: MUHAMMAD IHSAN
NIM	: 2025573010053
KELAS	: TI – 1C
MATA KULIAH	: PRAKTIKUM SISTEM OPERASI
PROGRAM STUDI	: TEKNIK INFORMATIKA
DOSEN PENGAMPU	: MUHAMMAD REZA ZULMAN,S.ST.,M.SC

PRODI TEKNIK INFORMATIKA

POLITEKNIK NEGERI LHOKSEUMAWE

TAHUN 2026/2027

LEMBAR PENGESAHAN

Nama	: Muhammad Ihsan
NIM	: 2025573010053
Kelas	: TI -1C
Jurusan	: Teknologi Informasi & Komputer
Prodi	: Teknik Informatika
Hari / Jam	: Selasa 22 April 2026 / 07.30 – 11.25
Dosen pengampu	: Muhammad Reza Zulman,S.ST.,M.SC

Disetujui,

Dosen Pengampu

Muhammad Reza Zulman,S.ST.,M.SC

KATA PENGANTAR

Puji dan syukur penulis panjatkan ke hadirat Tuhan Yang Maha Esa, karena atas rahmat dan karunia-Nya penulis dapat menyelesaikan laporan praktikum yang berjudul *"Analisis Kompleksitas Algoritma (Big O Notation, Time & Space Complexity)"* ini dengan baik.

Laporan ini disusun sebagai salah satu bentuk pemenuhan tugas pada mata kuliah Struktur Data dan Algoritma. Dalam laporan ini dibahas mengenai konsep dasar analisis kompleksitas algoritma, termasuk pengertian Big O Notation, time complexity, space complexity, serta penerapannya dalam berbagai contoh kode program menggunakan JavaScript.

Penulis menyadari bahwa laporan ini tidak akan terselesaikan dengan baik tanpa bantuan dan dukungan dari berbagai pihak. Oleh karena itu, penulis mengucapkan terima kasih kepada:

1. Dosen pengampu Bapak Muhammad Reza Zulman,S.ST.,M.SC mata kuliah Struktur Data dan Algoritma
2. Teman-teman yang telah membantu dalam proses praktikum
3. Semua pihak yang telah memberikan dukungan dan motivasi

Penulis menyadari bahwa laporan ini masih memiliki banyak kekurangan. Oleh karena itu, penulis sangat mengharapkan kritik dan saran yang membangun demi perbaikan di masa yang akan datang.

Akhir kata, penulis berharap semoga laporan ini dapat memberikan manfaat dan menambah wawasan bagi pembaca, khususnya dalam memahami analisis kompleksitas algoritma.

Buket Rata,22 April 2026

Muhammad Ihsan

DAFTAR ISI

LEMBAR PENGESAHAN ii
KATA PENGANTAR iii
DAFTAR ISI iv
BAB I 1
PENDAHULUAN 1
1.1 Latar Belakang 1
1.2 Tujuan Praktikum 1
BAB II 2
LANDASAN TEORI 2
2.1 Pengertian Analisis Kompleksitas Algoritma 2
2.2 Big O Notation 2
2.3 Time Complexity 3
2.4 Space Complexity 4
2.5 Time-Space Trade-off 5
2.6 Pola Umum Kompleksitas Algoritma 6
2.7 Pentingnya Analisis Kompleksitas 6
BAB III 7
METODOLOGI PRAKTIKUM 7
3.1 Waktu dan Tempat 7
3.2 Alat dan Bahan 7
3.3 Metode Praktikum 7
3.4 Prosedur / Langkah-langkah Praktikum 8
1. Langkah Pertama 8
2. Percobaan 1 – Mengukur Waktu Eksekusi 8
3. Percobaan 2 – Implementasi Kelas Big O 9
4. Percobaan 3 – Analisis Space Complexity 11
5. Percobaan 4 – Refactoring Algoritma 12
6. Pengerjaan Latihan 13
7. Pengerjaan Tugas 15
8. Dokumentasi dan Pengumpulan 17

BAB IV.....18
HASIL DAN PEMBAHASAN 18
4.1 Hasil Praktikum 18
4.2 Analisis.....19
BAB V.....20
PENUTUP 20
5.1 Kesimpulan.....20
5.2 Saran 20

BAB I

PENDAHULUAN

1.1 Latar Belakang

Dalam pengembangan perangkat lunak, efisiensi algoritma merupakan hal yang sangat penting. Algoritma yang tidak efisien dapat menyebabkan waktu eksekusi yang lama dan penggunaan memori yang besar, terutama ketika data yang diproses berukuran besar. Oleh karena itu, diperlukan suatu metode untuk menganalisis performa algoritma, yaitu menggunakan **Big O Notation**.

Big O digunakan untuk mengukur bagaimana waktu dan ruang (memori) yang dibutuhkan algoritma bertumbuh seiring dengan bertambahnya ukuran input. Dengan memahami konsep ini, programmer dapat menentukan apakah suatu algoritma sudah optimal atau masih bisa diperbaiki.

Berdasarkan modul praktikum, pemahaman ini menjadi dasar sebelum mempelajari struktur data lebih lanjut .

1.2 Tujuan Praktikum

Tujuan dari praktikum ini adalah:

1. Memahami pentingnya analisis efisiensi algoritma
2. Menjelaskan konsep **Time Complexity** dan **Space Complexity**
3. Memahami notasi Big O seperti O(1), O(n), O(n^2), dll
4. Menganalisis kompleksitas pada berbagai bentuk kode
5. Membandingkan performa algoritma secara empiris
6. Meningkatkan efisiensi algoritma melalui refactoring

BAB II

LANDASAN TEORI

2.1 Pengertian Analisis Kompleksitas Algoritma

Analisis kompleksitas algoritma merupakan suatu metode yang digunakan untuk mengukur tingkat efisiensi suatu algoritma dalam menyelesaikan masalah. Efisiensi ini dilihat dari dua aspek utama, yaitu waktu eksekusi (time complexity) dan penggunaan memori (space complexity).

Dalam dunia pemrograman, sebuah algoritma tidak hanya dinilai dari benar atau tidaknya hasil yang diberikan, tetapi juga dari seberapa cepat dan seberapa hemat sumber daya algoritma tersebut bekerja. Hal ini menjadi sangat penting terutama ketika algoritma digunakan untuk memproses data dalam jumlah besar.

Analisis kompleksitas membantu programmer dalam:

- Membandingkan dua atau lebih algoritma
- Menentukan algoritma yang paling optimal
- Menghindari penggunaan algoritma yang tidak efisien

2.2 Big O Notation

Big O Notation adalah notasi matematis yang digunakan untuk menyatakan batas atas (upper bound) dari pertumbuhan waktu atau ruang yang dibutuhkan oleh suatu algoritma terhadap ukuran input.

Big O berfokus pada **laju pertumbuhan (growth rate)**, bukan waktu sebenarnya. Hal ini dikarenakan waktu eksekusi nyata dipengaruhi oleh berbagai faktor seperti spesifikasi hardware, bahasa pemrograman, dan compiler yang digunakan.

Karakteristik Big O:

1. Mengabaikan konstanta

   Contoh:

   $2n \rightarrow O(n)$
2. Mengabaikan suku yang lebih kecil

   Contoh:

   $n^2 + n \rightarrow O(n^2)$
3. Fokus pada pertumbuhan saat n sangat besar

Dengan demikian, Big O memberikan gambaran umum tentang performa algoritma dalam kondisi terburuk (worst case).

2.3 Time Complexity

Time Complexity adalah ukuran yang menyatakan berapa lama waktu yang dibutuhkan suatu algoritma untuk menyelesaikan masalah berdasarkan ukuran input.

Jenis-jenis Time Complexity:

1. O(1) – Konstan

Waktu eksekusi tidak bergantung pada ukuran input.
Contoh:

- Mengakses elemen array berdasarkan indeks
- Operasi matematika sederhana

2. O(log n) – Logaritmik

Waktu bertambah secara lambat karena setiap langkah mengurangi ukuran masalah.
Contoh:

- Binary Search

3. O(n) – Linear

Waktu bertambah sebanding dengan jumlah data.
Contoh:

- Perulangan satu kali pada array

4. O(n log n) – Linearitmik

Gabungan antara linear dan logaritmik.
Contoh:

- Merge Sort
- Quick Sort

5. $O(n^2)$ – Kuadratik

Terjadi ketika terdapat perulangan bersarang (nested loop).
Contoh:

- Bubble Sort

6. $O(2^n)$ – Eksponensial

Pertumbuhan sangat cepat dan tidak efisien untuk data besar.
Contoh:

- Fibonacci rekursif

2.4 Space Complexity

Space Complexity adalah ukuran yang menunjukkan jumlah memori tambahan yang digunakan oleh suatu algoritma selama proses eksekusi.

Space complexity tidak menghitung ukuran input, melainkan hanya memori tambahan (auxiliary space).

Jenis Space Complexity:

1. **O(1) – Konstan**
   Menggunakan jumlah memori tetap
   Contoh: variabel sederhana
2. **O(n) – Linear**
   Memori bertambah sesuai ukuran input
   Contoh: membuat array baru
3. **O(n) pada Rekursi**
   Terjadi karena penggunaan call stack

2.5 Time-Space Trade-off

Dalam banyak kasus, terdapat hubungan antara waktu dan memori yang disebut **time-space trade-off**.

Artinya:

- Algoritma bisa dibuat lebih cepat dengan menggunakan lebih banyak memori
- Atau lebih hemat memori tetapi membutuhkan waktu lebih lama

Contoh:

- Fibonacci rekursif → lambat ($O(2^n)$) tetapi hemat memori
- Fibonacci dengan memoization → cepat ($O(n)$) tetapi menggunakan memori tambahan

2.6 Pola Umum Kompleksitas Algoritma

Beberapa pola kode yang sering digunakan dapat membantu dalam menentukan kompleksitas algoritma:

Pola Kode	Kompleksitas
Akses langsung (array/object)	O(1)
Satu perulangan	O(n)
Perulangan dengan pembagian dua (i *= 2)	O(log n)
Dua loop berurutan	O(n)
Nested loop	$O(n^2)$
Rekursi bercabang	$O(2^n)$

Dengan mengenali pola-pola ini, programmer dapat dengan cepat mengidentifikasi kompleksitas tanpa harus melakukan perhitungan yang rumit.

2.7 Pentingnya Analisis Kompleksitas

Analisis kompleksitas sangat penting dalam pengembangan perangkat lunak karena:

1. Membantu memilih algoritma terbaik
2. Menghindari performa yang buruk
3. Meningkatkan efisiensi aplikasi
4. Sangat penting untuk sistem dengan data besar (big data)

Sebagai contoh, algoritma dengan kompleksitas $O(n^2)$ mungkin masih bisa digunakan untuk data kecil, tetapi akan menjadi sangat lambat ketika data mencapai ribuan atau jutaan elemen.

BAB III

METODOLOGI PRAKTIKUM

3.1 Waktu dan Tempat

Praktikum ini dilaksanakan pada:

- **Hari/Tanggal** : Selasa 22 April 2026
- **Tempat** : Laboratorium Komputer 106
- **Waktu** : 7.30 – 11.25

3.2 Alat dan Bahan

Alat dan bahan yang digunakan dalam praktikum ini adalah:

1. Laptop/Komputer
2. Software **Node.js (v18 atau lebih baru)**
3. Text Editor (Visual Studio Code)
4. Terminal/Command Prompt
5. Modul praktikum Analisis Kompleksitas Algoritma

3.3 Metode Praktikum

Metode yang digunakan dalam praktikum ini adalah:

- **Metode Eksperimen**, yaitu melakukan percobaan langsung dengan menjalankan kode program
- **Metode Observasi**, yaitu mengamati hasil eksekusi program
- **Metode Analisis**, yaitu menganalisis kompleksitas algoritma berdasarkan hasil percobaan

3.4 Prosedur / Langkah-langkah Praktikum

1. Langkah Pertama

Langkah pertama adalah menyiapkan lingkungan kerja dengan membuat folder praktikum dan menginisialisasi project menggunakan Node.js:

- Membuka terminal
- Membuat folder praktikum-5 – `mkdir praktikum-5`
- Masuk ke folder tersebut – `cd praktikum-5`
- Menjalankan perintah `npm init -y`

2. Percobaan 1 – Mengukur Waktu Eksekusi

- Membuat file 01-intro-kompleksitas.js

JS 01-intro-kompleksitas.js

```javascript
// Fungsi bantu: dan waktu eksekusi
function ukurWaktu(label, fn) {
  const mulai = Date.now();
  fn();
  const selesai = Date.now();
  console.log("${label}: ${selesai - mulai} ms");
}
const N = 100_000; // ukuran data : 100.000

// --- Algoritma A: O(n) - loop satu kali ---
function jumlahkanLinear(n) {
  let total = 0;
  for (let i = 1; i <= n; i++) total += 1;
  return total;
}

// --- Algoritma B : O(1) - rumus matematika ---
function jumlahkanRumus(n) {
  return (n * (n + 1)) / 2;
}
```

```
const data = Array.from({ length: 5000 }, (_, i) => i);

console.log("=== Perbandingan Waktu Eksekusi ===");
ukurWaktu("O(1) jumlahkanRumus ", () => jumlahkanRumus(N));
ukurWaktu("O(n) jumlahkanLinear ", () => jumlahkanLinear(N));
ukurWaktu("O(n2) cariPasangan ", () => cariPasangan(data));

console.log("\nHasil sama?", jumlahkanLinear(100) === jumlahkanRumus(100));
```

- Menjalankan program dan mengamati hasilnya

```
=== Perbandingan Waktu Eksekusi ===
${label}: ${selesai - mulai} ms
${label}: ${selesai - mulai} ms
${label}: ${selesai - mulai} ms

Hasil sama? false
```

3. Percobaan 2 – Implementasi Kelas Big O

- Membuat file 02-kelas-big-o.js

JS 02-kelas-big-o.js

```
// ─── O(1) — Konstan ──────────────────────────────────
function ambilPertama(arr) {
  return arr[0];
} // O(1)
function tambahkanItem(arr, item) {
  arr.push(item);
} // O(1)
function isGenap(n) {
  return n % 2 === 0;
} // O(1)
// ─── O(log n) — Logaritmik ───────────────────────────
function binarySearch(arr, target) {
  let kiri = 0,
    kanan = arr.length - 1;
  let langkah = 0;
  while (kiri <= kanan) {
    langkah++;
    const tengah = Math.floor((kiri + kanan) / 2);
    if (arr[tengah] === target) {
      console.log(` Ditemukan di indeks ${tengah} setelah ${langkah} langkah`);
      return tengah;
    }
    if (arr[tengah] < target) kiri = tengah + 1;
    else kanan = tengah - 1;
  }
  return -1;
}
```

```javascript
function cariMax(arr) {
  let maks = arr[0];
  for (const x of arr) if (x > maks) maks = x;
  return maks;
}
// ─── O(n2) — Kuadratik ──────────────────────────────
function bubbleSortNaif(arr) {
  const a = [...arr];
  for (let i = 0; i < a.length; i++)
    for (let j = 0; j < a.length - i - 1; j++)
      if (a[j] > a[j + 1]) [a[j], a[j + 1]] = [a[j + 1], a[j]];
  return a;
}
// ─── O(2n) — Eksponensial ───────────────────────────
function fibRekursif(n) {
  if (n <= 1) return n;
  return fibRekursif(n - 1) + fibRekursif(n - 2); // dua panggilan rekursif!
}
// ── Demonstrasi ──────────────────────────────────────
console.log("=== O(1) — selalu cepat ===");
console.log(ambilPertama([10, 20, 30, 40, 50])); // 10
console.log(isGenap(42)); // true
console.log("\n=== O(log n) — Binary Search ===");
const sorted = Array.from({ length: 1_000_000 }, (_, i) => i);
binarySearch(sorted, 731_452); // cari di 1 juta data
console.log("\n=== O(n) — Linear Search ===");
console.log(
  "Max dari 1000 elemen:",
  cariMax(Array.from({ length: 1000 }, () => (Math.random() * 1000) | 0)),
);
console.log("\n=== O(n2) — Bubble Sort (kecil saja) ===");
console.log(bubbleSortNaif([64, 34, 25, 12, 22, 11, 90]));
console.log("\n=== O(2n) — Fibonacci Rekursif (n kecil!) ===");
for (let i = 0; i <= 10; i++) process.stdout.write(fibRekursif(i) + " ");
console.log("");
// Demonstrasi betapa lambatnya O(2n)
console.log("\nWaktu fib(35) O(2n):");
let t = Date.now();
fibRekursif(35);
console.log(Date.now() - t, "ms");
console.log("Waktu fib(35) O(n) memoization:");
const memo = {};
function fibMemo(n) {
  if (n <= 1) return n;
  if (memo[n]) return memo[n];
  return (memo[n] = fibMemo(n - 1) + fibMemo(n - 2));
}
t = Date.now();
fibMemo(35);
console.log(Date.now() - t, "ms");
```

- Menjalankan program dan mengamati hasilnya

```
=== O(1) — selalu cepat ===
10
true

=== O(log n) — Binary Search ===
 Ditemukan di indeks 731452 setelah 19 langkah

=== O(n) — Linear Search ===
Max dari 1000 elemen: 998

=== O(n2) — Bubble Sort (kecil saja) ===
[
  11, 12, 22, 25,
  34, 64, 90
]

=== O(2n) — Fibonacci Rekursif (n kecil!) ===
0 1 1 2 3 5 8 13 21 34 55

Waktu fib(35) O(2n):
116 ms
Waktu fib(35) O(n) memoization:
0 ms
```

4. Percobaan 3 – Analisis Space Complexity

- Membuat file 03-space-complexity.js

JS 03-space-complexity.js

```javascript
function jumlahArray(arr) {
  let total = 0; // hanya 1 variabel tambahan
  for (const x of arr) total += x;
  return total;
}
// --- O(n) Space: membuat struktur baru sebesar input ---
function duplikasiArray(arr) {
  const baru = []; // array baru tumbuh seiring arr
  for (const x of arr) baru.push(x * 2);
  return baru;
}
// --- O(n) Space: rekursi (call stack) ---
function faktorialRekursif(n) {
  if (n <= 1) return 1;
  return n * faktorialRekursif(n - 1); // n frame di call stack
}
// --- O(1) Space: faktorial iteratif ---
function faktorialIteratif(n) {
  let hasil = 1;
  for (let i = 2; i <= n; i++) hasil *= i; // hanya 2 variabel
  return hasil;
}
const arr = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10];
console.log("Jumlah array :", jumlahArray(arr)); // O(1) space
console.log("Duplikasi array:", duplikasiArray(arr)); // O(n) space
console.log("Faktorial 10 rekursif :", faktorialRekursif(10));
console.log("Faktorial 10 iteratif :", faktorialIteratif(10));
// Visualisasi: hitung elemen unik (O(n) space)
function hitungUnik(arr) {
  const seen = new Set(); // Set tumbuh hingga n elemenfor (const x of arr) seen.add(x);
  return seen.size;
}
const dataAcak = [1, 2, 3, 2, 1, 4, 5, 3, 6, 4, 7];
console.log("Elemen unik:", hitungUnik(dataAcak)); // 7

// 2. Fungsi mencari pasangan (nested loop)
// Big O Time: O(n^2)
// Alasan: ada 2 loop bersarang
// Big O Space: O(1)
// Alasan: tidak menggunakan struktur data tambahan
function cariPasanganLambat(arr, target) {
  for (let i = 0; i < arr.length; i++) {
    for (let j = i + 1; j < arr.length; j++) {
      if (arr[i] + arr[j] === target) {
        return [arr[i], arr[j]];
      }
    }
  }
  return null;
}
```

- Menjalankan program

```
Jumlah array : 55
Duplikasi array: [
   2,  4,  6,  8, 10,
  12, 14, 16, 18, 20
]
Faktorial 10 rekursif : 3628800
Faktorial 10 iteratif : 3628800
Elemen unik: 0
Hasil Lambat: [ 2, 7 ]
Hasil Cepat: [ 2, 7 ]
```

5. Percobaan 4 – Refactoring Algoritma

- Membuat file 04-refactor.js

JS 04-refactor.js

```javascript
// ==============================================
// SKENARIO 1: Cek duplikat dalam array
// ==============================================
// BURUK: O(n2) — nested loop
function adaDuplikatLambat(arr) {
  for (let i = 0; i < arr.length; i++)
    for (let j = i + 1; j < arr.length; j++) if (arr[i] === arr[j]) return true;
  return false;
}
// BAIK: O(n) — gunakan Set
function adaDuplikatCepat(arr) {
  const seen = new Set();
  for (const x of arr) {
    if (seen.has(x)) return true;
    seen.add(x);
  }
  return false;
}
// ==============================================
// SKENARIO 2: Frekuensi kemunculan elemen
// ==============================================
// BURUK: O(n2) — indexOf di dalam loop
function frekuensiLambat(arr) {
  const unik = [];
  const hitung = [];
  for (const x of arr) {
    const idx = unik.indexOf(x); // O(n) di dalam loop O(n)
    if (idx === -1) {
      unik.push(x);
      hitung.push(1);
    } else hitung[idx]++;
```

```
  }
  return Object.fromEntries(unik.map((u, i) => [u, hitung[i]]));
} // BAIK: O(n) — gunakan object/Map sebagai counter
function frekuensiCepat(arr) {
  const counter = {};
  for (const x of arr) counter[x] = (counter[x] || 0) + 1;
  return counter;
}
// ── Benchmark ──────────────────────────────
const besar = Array.from({ length: 20000 }, () =>
  Math.floor(Math.random() * 1000),
);
let t = Date.now();
adaDuplikatLambat(besar);
console.log("Duplikat LAMBAT O(n2):", Date.now() - t, "ms");
t = Date.now();
adaDuplikatCepat(besar);
console.log("Duplikat CEPAT O(n) :", Date.now() - t, "ms");
const kata = ["a", "b", "a", "c", "b", "a", "d"];
console.log("\nFrekuensi:", frekuensiCepat(kata));

```

- Membandingkan hasil sebelum dan sesudah optimasi

```
Duplikat LAMBAT O(n2): 0 ms
Duplikat CEPAT O(n) : 0 ms

Frekuensi: { a: 3, b: 2, c: 1, d: 1 }
```

6. Pengerjaan Latihan

- Membuat file latihan-1.js

JS latihan-1.js

```
// 3. Fungsi A
// Big O: O(1)
// Alasan: hanya melakukan 1 operasi (perkalian), tidak tergantung n
function fnA(n) {
  return n * 2;
}

// 4. Fungsi B
// Big O: O(n^2)
// Alasan: terdapat 2 loop bersarang, masing-masing berjalan n kali → n × n
function fnB(n) {
  for (let i = 0; i < n; i++) {
    for (let j = 0; j < n; j++) {
      // console.log(i, j); // dimatikan agar tidak memperlambat
    }
  }
```

```
}

// 5. Fungsi C
// Big O: O(log n)
// Alasan: nilai i dikali 2 setiap iterasi → jumlah iterasi logaritmik
function fnC(n) {
  for (let i = 1; i < n; i *= 2) {
    // console.log(i);
  }
}

// 6. Fungsi D
// Big O: O(n^3)
// Alasan: terdapat 3 loop bersarang → n × n × n
function fnD(arr) {
  arr.forEach((x) => {
    arr.forEach((y) => {
      arr.forEach((z) => {
        // console.log(x, y, z);
      });
    });
  });
}

// 8. Function untuk menghitung waktu eksekusi
function hitungKompleksitas(n, fn) {
  const start = Date.now();
  fn(n);
  const end = Date.now();
  console.log(`Waktu eksekusi: ${end - start} ms`);
}

// 7. Menjalankan fungsi dengan n = 1000
const n = 1000;

console.log("=== Fungsi A ===");
hitungKompleksitas(n, fnA);

console.log("=== Fungsi B ===");
hitungKompleksitas(n, fnB);

console.log("=== Fungsi C ===");
hitungKompleksitas(n, fnC);

// Untuk fungsi D menggunakan array
console.log("=== Fungsi D ===");
const arr = Array.from({ length: n }, (_, i) => i);

// Pengukuran khusus karena parameter berbeda
const startD = Date.now();
fnD(arr);
const endD = Date.now();
console.log(`Waktu eksekusi: ${endD - startD} ms`);

```

- Menjalankan program

```
=== Fungsi A ===
Waktu eksekusi: 0 ms
=== Fungsi B ===
Waktu eksekusi: 3 ms
=== Fungsi C ===
Waktu eksekusi: 0 ms
=== Fungsi D ===
Waktu eksekusi: 1116 ms
```

7. Pengerjaan Tugas

- Membuat folder tugas/
- Mengerjakan:
  - **Tugas 1**: Analisis dan refactor algoritma

JS tugas-1.js

```javascript
function intersectionLambat(arr1, arr2) {
  const hasil = [];

  for (let i = 0; i < arr1.length; i++) {
    for (let j = 0; j < arr2.length; j++) {
      if (arr1[i] === arr2[j] && !hasil.includes(arr1[i])) {
        hasil.push(arr1[i]);
      }
    }
  }

  return hasil;
}

// Versi cepat (pakai Set)
// Big O Time: O(n)
// Alasan: akses Set O(1), hanya 1 loop utama
// Big O Space: O(n)
function intersectionCepat(arr1, arr2) {
  const set2 = new Set(arr2);
  const hasil = [];

  for (let num of arr1) {
    if (set2.has(num) && !hasil.includes(num)) {
      hasil.push(num);
    }
  }

  return hasil;
}

// ==============================
// FUNGSI B - KELOMPOK ANAGRAM
// ==============================

// Big O Time: O(n * k log k)
// Alasan: n = jumlah kata, k = panjang kata (sorting tiap kata)
// Big O Space: O(n)
function kelompokAnagram(arr) {
  const map = {};

  for (let kata of arr) {
    const key = kata.split("").sort().join("");

    if (!map[key]) {
      map[key] = [];
    }

    map[key].push(kata);
  }

  return Object.values(map);
}

```

Hasil programnya

```
=== Fungsi A (Intersection) ===
Lambat: [ 3, 4, 5 ]
Cepat: [ 3, 4, 5 ]

=== Fungsi B (Anagram) ===
[ [ 'eat', 'tea', 'ate' ], [ 'tan', 'nat' ], [ 'bat' ] ]

=== Fungsi C (Triple) ===
Lambat: true
Cepat: true

=== UJI PERFORMA ===

-- Intersection --
Lambat: 76 ms
Cepat: 3 ms

-- Anagram --
Waktu: 4 ms

-- Triple --
Cepat: 2 ms
```

- **Tugas 2**: Benchmark pertumbuhan Big O

JS tugas-2.js

```javascript
function fn_O1(n) {
  return n + 1;
}

// O(n)
// Big O Time: O(n)
// Alasan: loop sebanyak n
function fn_On(n) {
  let sum = 0;
  for (let i = 0; i < n; i++) {
    sum += i;
  }
  return sum;
}

// O(n log n)
// Big O Time: O(n log n)
// Alasan: loop luar n, loop dalam log n
function fn_OnLogn(n) {
  let count = 0;

  for (let i = 0; i < n; i++) {
    for (let j = 1; j < n; j *= 2) {
      count++;
    }
  }

  return count;
}

```

- Menguji program dengan berbagai ukuran data

```
=== Ukuran n = 100 ===
O(1): 0 ms
O(n): 0 ms
O(n log n): 0 ms
O(n^2): 0 ms

=== Ukuran n = 500 ===
O(1): 0 ms
O(n): 0 ms
O(n log n): 0 ms
O(n^2): 1 ms

=== Ukuran n = 1000 ===
O(1): 0 ms
O(n): 0 ms
O(n log n): 0 ms
O(n^2): 3 ms

=== Ukuran n = 5000 ===
O(1): 0 ms
O(n): 0 ms
O(n log n): 1 ms
O(n^2): 93 ms

=== Ukuran n = 10000 ===
O(1): 0 ms
O(n): 0 ms
O(n log n): 0 ms
O(n^2): 193 ms
```

8. Dokumentasi dan Pengumpulan

- Mengecek seluruh file yang telah dibuat
- Menyimpan hasil praktikum
- Mengupload ke repository GitHub menggunakan perintah:
  - git add
  - git commit
  - git push

BAB IV

HASIL DAN PEMBAHASAN

4.1 Hasil Praktikum

Berdasarkan praktikum yang telah dilakukan mengenai analisis kompleksitas algoritma, diperoleh beberapa hasil sebagai berikut:

1. **Perbandingan Time Complexity**
   Dari hasil pengujian, algoritma dengan kompleksitas **O(1)** menunjukkan waktu eksekusi yang paling cepat karena tidak dipengaruhi oleh ukuran input. Sementara itu, algoritma **O(n)** membutuhkan waktu yang meningkat seiring bertambahnya jumlah data. Algoritma **$O(n^2)$** menunjukkan peningkatan waktu yang sangat signifikan karena adanya perulangan bersarang (nested loop).
2. **Pengaruh Ukuran Data terhadap Waktu Eksekusi**
   Semakin besar ukuran data yang digunakan, maka semakin terlihat perbedaan performa antar algoritma. Pada data kecil, perbedaan waktu tidak terlalu signifikan, namun pada data besar, algoritma dengan kompleksitas tinggi menjadi sangat lambat.
3. **Efisiensi Binary Search dibanding Linear Search**
   Hasil praktikum menunjukkan bahwa binary search jauh lebih efisien dibandingkan linear search. Hal ini karena binary search memiliki kompleksitas **O(log n)**, sedangkan linear search memiliki kompleksitas **O(n)**.
4. **Perbandingan Rekursi dan Iterasi**
   Algoritma rekursif, seperti Fibonacci, memiliki kompleksitas waktu yang lebih tinggi dibandingkan versi yang telah dioptimasi (memoization). Selain itu, rekursi juga membutuhkan memori lebih besar karena penggunaan call stack.
5. **Pengaruh Space Complexity**
   Algoritma yang menggunakan struktur data tambahan seperti array atau Set memiliki penggunaan memori yang lebih besar (**O(n)**), dibandingkan algoritma yang hanya menggunakan variabel sederhana (**O(1)**).
6. **Optimasi melalui Refactoring**
   Dengan melakukan refactoring dari nested loop ke penggunaan struktur data seperti **Set**

atau **Map**, kompleksitas algoritma dapat diturunkan dari **O(n^2)** menjadi **O(n)**. Hal ini terbukti meningkatkan performa secara signifikan.

4.2 Analisis

Berdasarkan hasil yang diperoleh, dapat disimpulkan bahwa pemilihan algoritma sangat mempengaruhi performa program. Algoritma yang sederhana belum tentu efisien, dan algoritma yang lebih optimal seringkali memerlukan pemahaman yang lebih mendalam.

Selain itu, terdapat hubungan erat antara waktu dan memori yang dikenal sebagai **time-space trade-off**, di mana peningkatan performa waktu sering kali membutuhkan tambahan penggunaan memori.

BAB V

PENUTUP

5.1 Kesimpulan

Berdasarkan praktikum yang telah dilakukan mengenai analisis kompleksitas algoritma, dapat disimpulkan bahwa:

1. **Big O Notation** merupakan metode yang penting untuk mengukur efisiensi algoritma berdasarkan pertumbuhan waktu dan memori terhadap ukuran input.
2. Algoritma dengan kompleksitas lebih rendah, seperti **O(1)** dan **O(log n)**, memiliki performa yang jauh lebih baik dibandingkan algoritma dengan kompleksitas tinggi seperti **$O(n^2)$** atau **$O(2^n)$**.
3. Perbedaan kompleksitas algoritma akan semakin terlihat signifikan ketika ukuran data yang diproses semakin besar.
4. **Time Complexity** dan **Space Complexity** harus dipertimbangkan secara bersamaan dalam menentukan algoritma yang optimal.
5. Penggunaan teknik optimasi seperti **refactoring** dan pemanfaatan struktur data yang tepat (misalnya Set atau Map) dapat meningkatkan efisiensi algoritma secara signifikan.
6. Analisis kompleksitas sangat membantu dalam memilih solusi terbaik sehingga program dapat berjalan lebih cepat dan efisien.

Dengan demikian, pemahaman tentang analisis kompleksitas algoritma menjadi dasar yang sangat penting dalam pengembangan perangkat lunak yang optimal dan efisien

5.2 Saran

Adapun saran yang dapat diberikan berdasarkan praktikum ini adalah:

1. Mahasiswa diharapkan lebih sering berlatih dalam menganalisis kompleksitas algoritma agar terbiasa mengenali pola-pola kompleksitas.
2. Dalam menulis program, sebaiknya tidak hanya fokus pada hasil, tetapi juga memperhatikan efisiensi algoritma yang digunakan.
3. Gunakan struktur data yang tepat seperti **Set** atau **Map** untuk meningkatkan performa program.

4. Hindari penggunaan nested loop jika masih dapat dioptimasi dengan pendekatan lain.
5. Lakukan pengujian performa secara empiris menggunakan tools seperti Date.now() untuk mengetahui efisiensi algoritma secara nyata.
6. Perlu adanya pemahaman lebih lanjut mengenai algoritma lanjutan agar dapat mengoptimalkan program secara maksimal.